

Anomaly Detection Via One-Class SVM

Yunu Choe

*dept. Electrical and Computer Engineering
University of Victoria
Victoria, Canada*

Abdel Yasser Mohamed

*dept. Electrical and Computer Engineering
University of Victoria
Victoria, Canada*

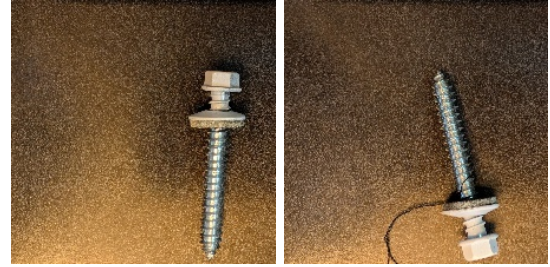
Abstract—This work seeks to investigate visual defects in images using anomaly detection methods, allowing for defects to be identified when they would otherwise cause harm. Various experiments using differing architectures were carried out to uncover the most robust, general solution. This pipeline involves a simple One-Class SVM alongside a pre-trained CNN, with EfficientNet-B0 for feature extraction and global pooling for dimensionality reduction. Therefore, this model leverages low training time and computation cost, while delivering consistent results for a multitude of differing image classes. Initially, two datasets of Screws and Pasta were used with our model, with the pair maintaining the same hyperparameters throughout. Once the results appeared satisfactory, a distinct third class of Capsules was introduced to test against overfitting. The SVM model showcased the ability to generalize regardless of the image type with an average AUROC score of 0.8087. Moreover, the standard deviation of AUROC scores between the three classes was 0.0440, further supporting this notion.

Keywords—Support Vector Machine, One-Class Classification, Feature Extraction, Dimensionality Reduction, Pooling, AUROC.

I. INTRODUCTION

The importance of anomaly detection is apparent when the presence of accurate and reliable images is required for a workflow of any kind [1]. Such instances include ID verification, video surveillance, and medical images. Thus, this remains a relevant field for applications of various computer vision and machine learning techniques [2]. This study will look exclusively at three classes of nominal images [3], before applying the mixed test batch to measure the success of the model's implementation.

Fig. 1 illustrates some examples of nominal and anomalous images for each of the three classes: Screws, Pasta, and Capsules. Note that there were 18, 17, and 20 images for training, respectively. The nominal images are visually homogeneous, featuring various focused views of the class. This is what we are training the model on, with the idea being that if a never before seen feature is present in the test, then by definition, it must be anomalous. Notice that the anomalous screw image has a wire-like object attached. Next, it is observed with the pasta images that there is a foreign object in the bottom corner of the picture. Finally, the anomalous capsule image has the right half completely crushed. These are some of the more apparent examples of anomalies and are quite easily identified by human sight and intuition, but remain steadfast challenges for machine.



(a) Screws: Nominal (L), Anomalous (R)



(b) Pasta: Nominal (L), Anomalous (R)



(c) Capsules: Nominal (L), Anomalous (R)

Fig. 1: Nominal and anomalous examples for each image class

Through various implementations, we will first trial different methods against one another. Afterwards, with a single technique in mind, the hyperparameter will be tuned to allow for the trained model to identify details needed for a robust solution. It is important to reiterate that the goal is not to make the best detection method for each class, but to design an implementation that can potentially be extended to any image class. This is the concept of generalization, which is foundational in machine learning to prevent the overfitting of data. As training one image class is relatively easy once a structure is built, the most challenging aspect of this project has been to achieve this generalization.

II. LITERATURE REVIEW

Several existing approaches to image-based anomaly detection were investigated before selecting a final method, each evaluated against the constraints of this project: nominal training sets of approximately 17–20 images per class, a Google Colab execution environment, and the requirement to generalize to an unseen dataset.

Clustering-based methods [4] group nominal feature representations into clusters and score test samples by their distance from cluster centers or by relative cluster size, flagging distant or isolated samples as anomalous. While intuitive, these methods require careful tuning of cluster count and distance thresholds that tends to be dataset-specific, making them poorly suited to a multi-dataset generalization requirement.

AnoDDPM [5] detects anomalies by training a denoising diffusion probabilistic model on nominal images, then measuring reconstruction inconsistencies at test time. Anomalous regions cannot be faithfully restored by the diffusion process. Despite offering both detection and spatial localization, the method was not pursued. Iterative inference across hundreds of denoising steps is computationally prohibitive in Colab, and fewer than 20 training images per class is insufficient to train a diffusion model without memorization.

FastFlow [6] was implemented and evaluated directly. A frozen pretrained ResNet-18 backbone extracts spatial patch-level feature vectors, and a normalizing flow network trained on nominal patches learns an invertible mapping f to a standard Gaussian, scored by the change-of-variables log-likelihood

$$\log p_X(\mathbf{x}) = \log p_Z(f(\mathbf{x})) + \log |\det J_f(\mathbf{x})| \quad (1)$$

Anomaly scores are assigned per patch and assembled into a localization heatmap. Our implementation achieved AUROC scores of 0.72, 0.77, and 0.80 across the three datasets, however the large parameter count of the flow network relative to the training set size produced poorly calibrated scores, with some nominal images outscoring anomalous ones.

One-Class SVM (OC-SVM) [3] learns a kernel-induced hyperplane enclosing the nominal training data, using the signed distance to that boundary as the anomaly score. Paired with an EfficientNet-B0 feature extractor, adaptive average pooling, and standard scaling, the OC-SVM fits a decision boundary in a fixed, rich feature space rather than optimizing new parameters from scratch. This makes it far more sample-efficient than FastFlow under our data constraints, and was selected as the final method.

III. PROPOSED APPROACH

Our anomaly detection pipeline treats the problem as one-class classification. During training, only nominal images are used to model the distribution of normal data in a learned feature space. At inference time, a test image is mapped into the same feature space and assigned an anomaly score based on its distance from the learned normal region. The full pipeline is: Image $\rightarrow$ preprocessing $\rightarrow$ EfficientNet-B0

embedding $\rightarrow$ feature compression $\rightarrow$ standardization $\rightarrow$ One-Class SVM $\rightarrow$ anomaly score.

Rather than training a deep classifier from scratch, our method uses a pretrained convolutional neural network as a fixed feature extractor and a classical one-class model as the final decision stage. This design is well suited to the project setting, where only a small number of nominal training images are available and anomalous training examples are not provided.

A. Input Preprocessing

Each input image is first converted to RGB tensor format, resized to 224×224 , and normalized using the ImageNet channel statistics. Let the input image be denoted by

$$x \in \mathbb{R}^{H \times W \times 3} \quad (2)$$

After preprocessing, the image is transformed into

$$\tilde{x} \in \mathbb{R}^{3 \times 224 \times 224} \quad (3)$$

The resize step ensures a consistent input size for the pretrained network, while normalization aligns the input distribution with the data distribution used during ImageNet pretraining.

B. Feature Extraction with EfficientNet-B0

To obtain a compact semantic representation of each image, we use EfficientNet-B0 as a pretrained feature extractor [7]. EfficientNet-B0 is a convolutional neural network trained on ImageNet, and in our implementation the final classification head is removed. Therefore, the network is not used to predict ImageNet classes; instead, it is used only to transform an image into a high-level feature embedding.

Let the truncated EfficientNet backbone be represented by the function

$$\phi(\tilde{x}) \quad (4)$$

For each preprocessed image $\tilde{x}$, the backbone produces a feature tensor

$$F = \phi(\tilde{x}) \in \mathbb{R}^{1280 \times 1 \times 1} \quad (5)$$

In this 1280-dimensional feature vector, each dimension represents the activation of a learned channel in the final backbone layer. These features are not hand-designed descriptors; they are high-level visual features learned during large-scale supervised pretraining. In practice, this embedding captures useful information about texture, local shape, appearance consistency, and overall visual structure.

C. Feature Compression

The raw EfficientNet embedding has dimensionality 1280, which is relatively large compared to the small number of training images available in this project. To reduce dimensionality before fitting the One-Class SVM, our implementation compresses the 1280-dimensional feature vector using adaptive average pooling.

Let

$$f \in \mathbb{R}^{1280} \quad (6)$$

denote the flattened EfficientNet feature vector. We transform f into a lower-dimensional vector

$$z \in \mathbb{R}^d \quad (7)$$

where $d \in \{32, 56, 64, 128\}$ is treated as a hyperparameter.

In our implementation, this compression reduces its width from 1280 to d . Conceptually, this operation partitions the original embedding into d bins and averages the activations within each bin. For example, if $d = 64$, each compressed feature summarizes approximately 20 original feature values.

D. Feature Standardization

After compression, the feature vectors are standardized before being passed to the One-Class SVM. The SVM depends on Euclidean distances in feature space. Without standardization, features with larger numerical magnitudes would dominate the distance calculation. Let the compressed feature vector be

$$z = [z_1, z_2, \dots, z_d]^T \quad (8)$$

For each feature dimension j , standardization is performed as

$$\hat{z}_j = \frac{z_j - \mu_j}{\sigma_j} \quad (9)$$

where μ_j and σ_j are the mean and standard deviation of feature j computed over the nominal training set.

E. One-Class SVM in Feature Space

Once a standardized feature representation has been computed for each nominal training image, a One-Class SVM is used to learn the support of the normal-data distribution. The SVM is trained only on nominal samples and attempts to define a boundary enclosing the region of feature space associated with normal images.

Given a test feature vector x , the model evaluates its similarity to the learned support vectors using the radial basis function (RBF) kernel

$$K(x_i, x) = \exp(-\gamma \|x_i - x\|^2) \quad (10)$$

where x_i is a support vector, $\|\cdot\|$ denotes Euclidean distance, and $\gamma > 0$ controls how quickly similarity decays with distance.

In our implementation, `gamma='scale'` is used. In scikit-learn [8], this corresponds to

$$\gamma = \frac{1}{d \cdot \text{Var}(X)} \quad (11)$$

where d is the number of standardized input features and $\text{Var}(X)$ is the variance of the feature matrix received by the SVM. Since the features are standardized before classification, this typically results in a reasonable default kernel width without manual tuning.

The One-Class SVM decision function is

$$f(x) = \sum_{i=1}^m \alpha_i K(x_i, x) - \rho \quad (12)$$

where:

- x_i are the learned support vectors,
- α_i are the learned coefficients,
- m is the number of support vectors,
- ρ is the learned offset term.

A positive value of $f(x)$ indicates that the sample lies inside the learned normal region, while a negative value indicates that it lies outside the boundary and is therefore more likely to be anomalous.

The hyperparameter ν controls the softness of the learned boundary. In a One-Class SVM, ν acts as an upper bound on the fraction of training errors and a lower bound on the fraction of support vectors. Intuitively, larger ν values allow a looser boundary that tolerates more training irregularities, while smaller ν values encourage a tighter description of normality.

For anomaly detection, it is more intuitive to use larger values for more anomalous samples. Therefore, our implementation defines the anomaly score as

$$s(x) = -f(x) \quad (13)$$

This sign inversion does not change the ranking of samples, but it makes interpretation more intuitive. Images with larger $s(x)$ are considered more anomalous, while smaller values indicate samples that are closer to the learned nominal distribution.

IV. RESULTS AND EVALUATION

As the *pooling dimensions* became the sole impactful hyperparameter for our final implementation, tuning it with various values became the key to discovering the most consistent result. As mentioned above, *nu* and *gamma* remained fixed at 0.01 and 'scale' respectively. Table I details the various pooling dimensions and the respective AUROC scores, the average and standard deviation of the AUROC scores.

TABLE I: Hyperparameter Search for Pooling Dimension

Pool Dim	Screws	Pasta	Capsule	Avg AUROC	Std Dev
32	0.6800	0.9167	0.7124	0.7697	0.1048
56	0.7600	0.8667	0.7993	0.8087	0.0440
64	0.7400	0.8667	0.7625	0.7897	0.0552
128	0.6800	0.8667	0.8027	0.7831	0.0775

It is observed that the Pasta class achieves the highest AUROC score of 0.9167 at the lowest pooling dimension. Conversely, the Capsule class performed best at the highest pooling dimension. The Screw class presented itself as the most challenging class to work with, and it achieved its peak with the intermediate pooling dimension values. Here, the other classes suffered only marginally, which made 64 a reasonable middle ground. Upon further refinement, it was discovered that a pooling dimension of 56 provided the greatest balance. This is demonstrated by the single highest average AUROC value, which slightly exceeded a value of 0.8. Moreover, it can also be noted that the Capsule class performed only marginally below its optimum. While the Pasta class saw a decrease, it remained relatively stable compared to other non-optimal pooling dimensions. The high performance and robustness of this hyperparameter value is further supported by the standard deviation. While this alone represents little, compounded with the high AUROC score, it showcases that each class has results with little deviation and high generalization. Moreover, since the Capsule class was introduced at the latter stages of this project, with less time spent tuning the underlying SVM architecture. Despite this pseudo-blind testing, it still outperformed the Screw class, which comparatively received more optimization.

Furthermore, Table II captures additional details regarding the optimal solution, with additional details such as accuracy, F1 score and the resultant confusion matrix.

Note that the confusion matrix follows the convention of:

$$\begin{bmatrix} TP & FN \\ FP & TN \end{bmatrix}$$

TABLE II: Final Performance Metrics (Pooling Dimension = 56)

Dataset	AUROC	Accuracy	F1 Score	Confusion Matrix	
Screws	0.7600	0.6667	0.6667	5	5
Pasta	0.8667	0.6875	0.6667	5	5
Capsule	0.7993	0.6944	0.6667	11	2
				9	14

This further showcases the underlying merits and drawbacks of the SVM implementation. The high AUROC score remains as presented before, but the introduction of the confusion matrices, along with the accuracy and F1 scores, highlights a notable limitation. In isolation, the AUROC score is quite solid across the board and signifies a good model, as corroborated by the ROC curves in Figure 4. However, when compared to the confusion matrix, which directly measures the mistakes, the false positives and the false negatives, we can see that under a binary threshold, the model does not deliver. Since pure random guesses result in an accuracy of 0.5, our range of 0.67 to 0.69 is only a small improvement. Thus, the problem likely lies in the threshold value of 0.5 being set for the anomaly score as in many cases, the anomaly scores border

the decision boundary closely. This introduces the problem in which a binary result becomes misleading, which is visually evident in the boundary scores shown in Figure 2.

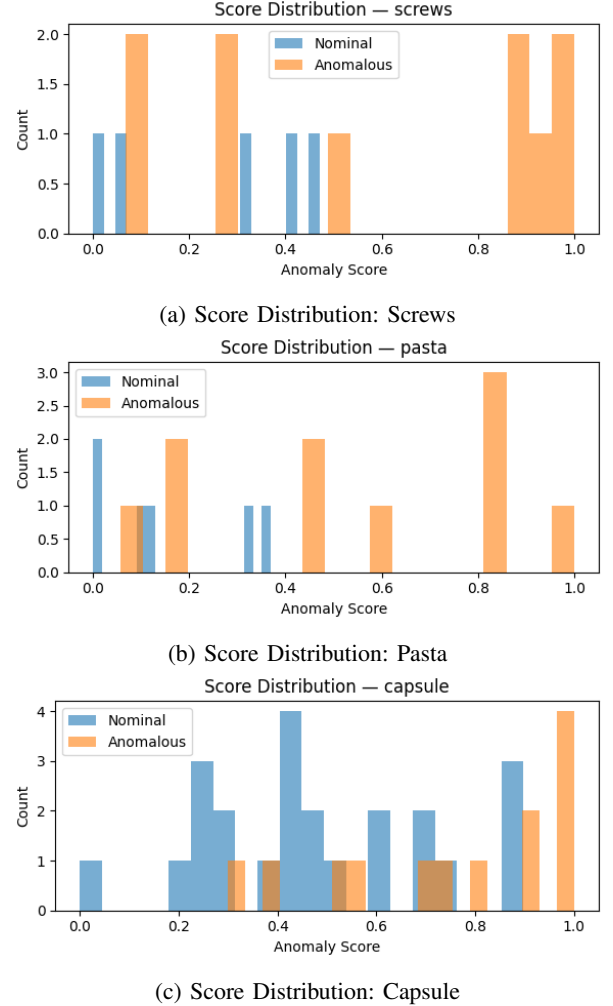
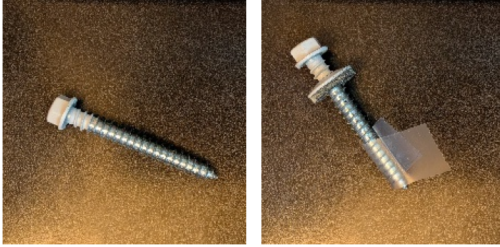


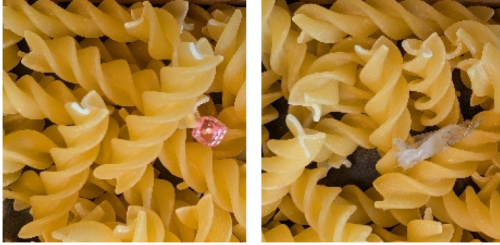
Fig. 2: Anomaly score histogram

A perfect implementation would result in the confusion matrix having diagonals with all other values being zero. However, it is clear that the Screws and Pasta datasets have many false negatives, while the Capsule dataset has many false positives. This introduces a clear dichotomy. The mistakes are not random, but are directly related to the classes. Let us first consider why the false negatives are so prevalent for Screws and Pasta. This limitation can largely be attributed to the underlying architecture of our design. Dimension pooling reduced the EfficientNet features to just 56 dimensions, meaning that small details were not fully captured. As demonstrated by Fig. 3, misclassified samples for the Screw dataset typically involved missing washers or the presence of attached foreign material. Furthermore, misclassifications for the Pasta dataset were often associated with visually distinct foreign objects embedded among otherwise nominal pasta pieces. This helps explain why there were zero false positives for both Screw

and Pasta, because only the presence of an anomaly throws off the model that was trained on a small sample of nominal images. Secondly, let us consider why the false positives primarily affected the Capsule dataset. In contrast to the other classes, the case of anomalous objects are not quite as clear cut as a foreign object being present. Instead, simple orientation changes of the capsule are enough to mimic an anomaly. Because the training set was so small, the model is accustomed to seeing a certain style of writing in certain locations. Deviating from this strict visual baseline causes the model to predict that it has discovered an anomaly, rather than a simple change in viewpoint. Thus, it clarifies why the three classes failed in the way that they did.



(a) Anomalous Screw samples misclassified



(b) Anomalous Pasta samples misclassified



(c) Nominal Capsule samples misclassified

Fig. 3: Incorrectly classified samples

Finally, having discussed the AUROC score and the confusion matrix, it is paramount to understand why the seemingly contradictory scores occurred, and why it does not invalidate the findings. The AUROC score circumvents the raw accuracy score and its limitations by evaluating the model's ability to rank anomalies across all possible decision boundaries. Consequently, the AUROC score provides a far more robust metric for measuring the learning and generalization capabilities of this pipeline. Although the current binary implementation yields suboptimal accuracy, the high AUROC indicates that the underlying feature extraction is successfully separating

nominal and anomalous data. For example, with a larger dataset to better capture class variance, the binary accuracy would likely align more closely with the model's demonstrated learning capacity.

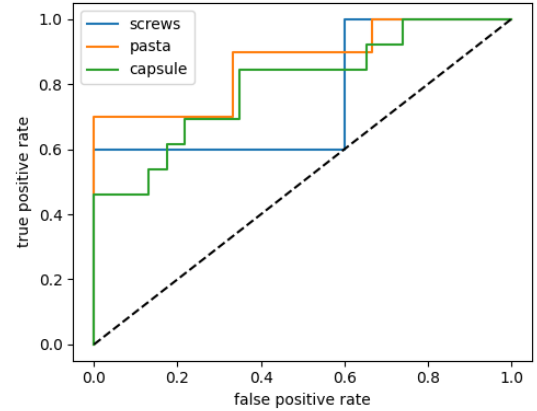


Fig. 4: Combined ROC curves for all three datasets

V. CONCLUSION

The findings from this project demonstrate that an effective, general classical machine learning method was successfully implemented to detect anomalous images. Providing a relatively small number of nominal training images proved to be more than satisfactory, with a tuned average AUROC score of 0.8087 and a standard deviation of 0.0440. By first considering a multitude of other potential solutions and then committing to the SVM, we can further confirm that the results were the product of testing and validation. Moreover, the tuning of various parameters, even if only one mattered in the end, agrees with this idea. Given the constraints of a small dataset, the SVM performed optimally.

REFERENCES

- [1] S. K. Adari and S. Alla, *Beginning Anomaly Detection Using Python-Based Deep Learning: Implement Anomaly Detection Applications with Keras and PyTorch*, 2nd ed. Berkeley, CA: Apress, 2024.
- [2] A. Albu, "Project Document ECE 471 Spring 2026," University of Victoria – Department of Electrical and Computer Engineering, Victoria, BC, Canada, 2026.
- [3] MVTec Software GmbH, "Machine Vision Datasets for Research — MVTec," <https://www.mvtec.com/company/research/datasets/mvtec-ad>, accessed: Mar. 26, 2026.
- [4] G. Pang, C. Shen, L. Cao, and A. V. D. Hengel, "Deep Learning for Anomaly Detection: A Review," *ACM Computing Surveys*, vol. 54, no. 2, 2022.
- [5] J. Wyatt, A. Leach, S. M. Schmon, and C. G. Willcocks, "AnoDDPM: Anomaly Detection with Denoising Diffusion Probabilistic Models using Simplex Noise," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*. IEEE, 2022, pp. 649–655.
- [6] T. Yu, X. Li, Y. Chen, and C. Shen, "FastFlow: Unsupervised Anomaly Detection and Localization via 2D Normalizing Flows," *arXiv*, 2021.
- [7] S. Li, Y. Zhao, R. Varma, O. Salpekar *et al.*, "PyTorch Distributed: Experiences on Accelerating Data Parallel Training," 2020. [Online]. Available: <https://arxiv.org/abs/2006.15704>
- [8] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.